I am asking *view* to listen to the button click event and save the book when such an event occurs:

```
<script type="text/template" id="book_template">

       <label>Book Details</label>
       <input type="text" id="book_title" value=" <%= title
%> "/>
       <input type="button" id="book_author" value="<%=
author %> " />
       <input type="button" id="save_button" value="save" />
</script>

<div id="book_container"></div>

<script type="text/javascript">
      BookView = Backbone.View.extend({
          initialize: function(){
             this.render();
          },
         render: function(){
            /* Compile the template using underscore */
          var template = _.template( $("#book_template").
html(), this.model.toJSON()
);
        /* Load the compiled HTML into the Backbone "el" */
       this.$el.html(template );
      },

  events: {
          "click input[type=button]": "saveBook"
           },
saveBook: function( event ){
 this.model.set(
{
title : $("#book_title").val(),
author : $("#book_author").val()
}
);
this.model.save();
}
     }

  });

var myBook = new Book(
{

   title : 'Open Source',
   author : 'Pradeep Meruva',
   genre : 'Technical'
});
     var book_view = new BookView({ el: $("#book_container"),
model : myBook });
</script>
-
```

Router

Web applications often provide linkable, bookmarkable, shareable URLs for important locations in the app. Until recently, hash fragments (#page) were used to provide these permalinks, but with the arrival of the History API, it's now possible to use standard URLs (/page). Backbone.Router provides methods for routing client-side pages, and connecting them to actions and events. For browsers that don't yet support the History API, the router handles graceful fallback and transparent translation to the fragment version of the URL.

During page load, after your application has finished creating all of its routers, be sure to call *Backbone.history.start()* or *Backbone.history.start({pushState: true})* to route the initial URL.

```
var Workspace = Backbone.Router.extend({

  routes: {

  "help":             "help",  // #help

  "search/:query":  "search", // #search/kiwis

  "search/:query/p:page": "search"   // #search/kiwis/p7

  },

  help: function() {

    ...

  },

  search: function(query, page) {
  ...

  }

});
```

References

[1] *http://backbonejs.org/*
[2] *http://backbonetutorials.com/*

By: Pradeep Meruva

The author is a solutions architect at Ericsson. He has been working with open source technologies for over 10 years. After being in a decade long relationship with Java-related technologies, he recently started dating technologies related to JavaScript. His interest lies in developing scalable Web applications with open source technologies. He can be reached at *pradeep.meruva@gmail.com*.